

Capítulo

2

Sistema de Gestão de Extensão do IC: Rotinas Avançadas (OCRA)

Arthur Schineider, Felipe Mello, Felipe Texeira, Jaiana Santana e Márcio Andrade

Abstract

This report describes the development of the artifacts for Marco 2 (OCRA) of the Extension Management System of the Instituto de Computação at UFBA, expanding the relational database built in Delivery 1 with new functionalities. Two Stored Procedures and two explicit transactions with atomicity control via PL/pgSQL were implemented, along with 10 Materialized Views to support strategic and operational analytical dashboards. Privacy policies were reinforced through Data Masking and role-based access control (RBAC), in compliance with the LGPD. Additionally, inconsistencies identified in Delivery 1 were corrected, covering the logical model, benchmark results, MAD1 naming conventions, security policies, and analytical queries.

Resumo

Este relatório descreve o desenvolvimento dos artefatos do Marco 2 (OCRA) do Sistema de Gestão de Extensão do Instituto de Computação da UFBA, expandindo o banco de dados relacional da Entrega 1 com novas funcionalidades. Foram implementadas 2 Stored Procedures e 2 transações explícitas com controle de atomicidade via PL/pgSQL, além de 10 Materialized Views para suporte a dashboards analíticos estratégicos e operacionais. As políticas de privacidade foram reforçadas com Data Masking e controle de acesso baseado em perfis (RBAC), em conformidade com a LGPD. Adicionalmente, foram corrigidas inconsistências identificadas na Entrega 1, abrangendo modelo lógico, benchmark, nomenclatura MAD1, políticas de segurança e consultas analíticas.

2.1. Descritivo do Projeto

Na Entrega 1 (Marco 1 — MIBD), foi modelado e implantado um banco de dados relacional para gerenciar os projetos de extensão, instrutores, participantes e atividades do

Instituto de Computação, com suporte a 30 consultas analíticas, plano de indexação e políticas de governança. O modelo se mostrou robusto e não demandou alterações estruturais no esquema conceitual.

Para esta Entrega 2 (Marco 2 — OCRA), o banco foi expandido com suporte transacional às principais operações do sistema, *Views* Materializadas para otimização de dashboards analíticos, mecanismos de proteção de privacidade (*Data Masking*) e rotinas de backup.

2.1.1. Alterações em relação à Entrega 1

A partir da revisão do material produzido na Entrega 1, foram identificadas e corrigidas inconsistências no modelo, no script DDL, nas políticas de segurança e nas consultas desenvolvidas. As alterações foram agrupadas por categoria e descritas a seguir.

2.1.1.1. Correção do Relatório da Entrega 1

As seguintes inconsistências foram identificadas e corrigidas diretamente no relatório da Entrega 1:

Modelo Lógico O modelo lógico relacional, que havia sido omitido na primeira versão do relatório, foi desenvolvido e incorporado à versão revisada da Entrega 1.

Consultas e Requisitos As justificativas das consultas em relação aos requisitos funcionais foram revisadas, assim como a justificativa dos próprios requisitos em relação ao minimundo do sistema.

Benchmark Os dados de desempenho reportados na primeira versão estavam incorretos devido a falhas metodológicas na coleta. Os resultados foram revalidados e corrigidos na seção de Plano de Indexação.

2.1.1.2. Script DDL e População

Restrição de unicidade no e-mail Foi adicionada a restrição `UNIQUE` na coluna `DS_EMAIL_CONTATO` da tabela `TB_PARTICIPANTE`, garantindo que dois participantes não possam ser cadastrados com o mesmo endereço de e-mail.

Check Constraints em colunas numéricas Foram adicionadas restrições `CHECK` nas colunas `VL_CARGA_HORARIA` (`RL_ALOCACAO_INSTRUTOR`) e `VL_APORTE` (`RL_PATROCINIO_EVENTO`) para impedir a inserção de valores negativos, garantindo a integridade dos dados numéricos.

ON DELETE SET NULL em ID_PROJ_VINCULADO A chave estrangeira que vincula uma atividade ao seu projeto pai (`ID_PROJ_VINCULADO`) passou a utilizar a cláusula `ON DELETE SET NULL`. Essa decisão se justifica pelo fato de que projetos

de extensão podem ser extintos ou ter seus registros removidos do banco. A utilização do comportamento padrão `RESTRICT` impossibilitaria a exclusão de um projeto enquanto houvesse atividades vinculadas a ele no histórico, comprometendo a flexibilidade operacional do sistema.

ON CONFLICT DO NOTHING na população Foi adicionada a cláusula `ON CONFLICT DO NOTHING` no script de população da tabela `RL_INSCRICAO_HISTORICO`, evitando que a restrição `UNIQUE` sobre o par (`ID_PARTICIPANTE`, `ID_ATIVIDADE`) interrompesse a execução do script ao encontrar tentativas de inscrições duplicadas.

2.1.1.3. Nomenclatura conforme MAD1

Correção de `DH_OPERACAO` A coluna `DH_OPERACAO` das tabelas de auditoria foi renomeada para `DT_OPERACAO`, em conformidade com a exigência estrita da sigla `DT_` definida no Documento de Arquitetura MAD1 para colunas de data e hora.

Desmembramento das Triggers As *triggers* aglutinadas (ex: `TG_A_IUD_TB_ATIVIDADE`), que tratavam as três operações em um único artefato, foram desmembradas em três *triggers* separadas por operação (ex: `TG_A_I_TB_ATIVIDADE`, `TG_A_U_TB_ATIVIDADE` e `TG_A_D_TB_ATIVIDADE`), adequando a nomenclatura ao padrão MAD1 e tornando o controle de auditoria mais granular.

Renomeação das Chaves Estrangeiras As *constraints* de chave estrangeira foram renomeadas para refletir a semântica completa exigida pelo padrão MAD1, substituindo nomenclaturas genéricas por nomes que identificam explicitamente as tabelas de origem e destino do relacionamento.

2.1.1.4. Políticas de Segurança

Renomeação do perfil de backup O perfil de backup foi renomeado para `dbbackup_ic`, substituindo a nomenclatura anterior que utilizava o prefixo `pg_`. O PostgreSQL reserva esse prefixo exclusivamente para funções internas e contas de administração do próprio sistema, bloqueando a criação de *roles* com essa nomenclatura.

Privilégios do perfil `dbbackup_ic` Foi identificado um erro no script DCL em que os privilégios do usuário de backup (`dbbackup_ic`) estavam sendo indevidamente revogados (`REVOKE`) nas tabelas de auditoria. O erro foi corrigido, garantindo que o perfil mantenha o acesso de leitura necessário para a execução dos backups.

Remoção do `ALTER DEFAULT PRIVILEGES` Foi identificado um erro de segurança no script DCL, onde o uso de `ALTER DEFAULT PRIVILEGES` concedia acesso DML ao perfil `sistema_ic` de forma irrestrita para todas as novas tabelas criadas no esquema `public`. O trecho foi removido pois esse comando aplica os privilégios automaticamente a qualquer tabela criada futuramente no esquema, violando

o princípio do menor privilégio e comprometendo o controle granular de acesso definido na política PPP1.

2.1.1.5. Plano de Indexação

O plano de indexação foi revisado e reduzido de 10 para 7 índices. Três índices planejados anteriormente foram dispensados por redundância com restrições UNIQUE já existentes: TB_EMISSAO_CERTIFICADO (ID_INSCRICAO), TB_REGISTRO_FEEDBACK (ID_INSCRICAO) e TB_PARTICIPANTE (DS_EMAIL_CONTATO) já possuem índices B-Tree criados automaticamente por suas respectivas constraints, dispensando a criação manual. Os 7 índices mantidos estão organizados em três categorias:

Índices para chaves estrangeiras aceleram os *JOINS* nas consultas:

- IDX_PROJETO_COORDENADOR em TB_PROJETO_EXTENSAO (ID_INSTR_COORDENADOR)
- IDX_ATIVIDADE_PROJETO em TB_ATIVIDADE (ID_PROJ_VINCULADO)
- IDX_INSCRICAO_PARTICIPANTE em RL_INSCRICAO_HISTORICO (ID_PARTICIPANTE)
- IDX_INSCRICAO_ATIVIDADE em RL_INSCRICAO_HISTORICO (ID_ATIVIDADE)

Índices para filtros aceleram cláusulas WHERE e GROUP BY:

- IDX_INSCRICAO_PRESENCA em RL_INSCRICAO_HISTORICO (ST_PRESENCA)
- IDX_ATIVIDADE_DATA em TB_ATIVIDADE (DT_REALIZACAO)

Índice composto otimiza consultas analíticas que filtram por presença e calculam agregações sobre notas simultaneamente:

- IDX_INSCRICAO_PRESENCA_NOTA em RL_INSCRICAO_HISTORICO (ST_PRESENCA, VL_NOTA_AVALIACAO)

2.1.1.6. Consultas Desenvolvidas

Correção das consultas I5 e I10 As consultas I5 e I10 estavam omitindo dados por não agregarem corretamente valores textuais em grupos. A função STRING_AGG foi aplicada para consolidar esses valores, corrigindo o comportamento das consultas.

Correção das consultas A13 e A15 As consultas A13 e A15 foram revisadas pois retornavam resultados vazios. A causa foi identificada como filtros excessivamente restritivos em relação ao volume de dados sintéticos populados no banco. Os critérios de filtragem foram ajustados para refletir corretamente a intenção analítica de cada consulta.

2.2. Artefatos de Sistema de Informação

Conforme exigido pelo Quadro 3, foram elaboradas rotinas em SQL (*Stored Procedures*, Transações e *Materialized Views*) para apoiar os artefatos do Sistema de Informação. A tabela a seguir apresenta o mapeamento entre os artefatos desenvolvidos e os requisitos funcionais atendidos.

Tabela 2.1. Mapeamento de artefatos e requisitos

Artefato	Nome	Req.
Stored Procedure	SP_CADASTRAR_PARTICIPANTE_COMPLETO	RF2
Stored Procedure	SP_GERENCIAR_ATIVIDADE	RF1
Transação	SP_INSCREVER_COM_VALIDACAO	RF2
Transação	SP_TRANSFERIR_PARTICIPANTE	RF7
Materialized View	VM_INDICE_RETENCAO_PROJETO	RF2
Materialized View	VM_GAP_EMISSAO_CERTIFICADO	RF3
Materialized View	VM_DISTRIBUICAO_QUARTIS_NOTAS	RF6
Materialized View	VM_COBERTURA_INSTRUTORES_PROJETO	RF1
Materialized View	VM_COHORT_PRIMEIRA_INSCRICAO	RF2
Materialized View	VM_NPS_POR_PROJETO	RF4
Materialized View	VM_MATRIZ_VINCULO_PRESENCA	RF2
Materialized View	VM_IMPACTO_PATROCINIO_ATIVIDADE	RF5
Materialized View	VM_VOLUME_SEMANAL_INSCRICOES	RF2
Materialized View	VM_ROI_PARCEIROS	RF5

2.2.1. Tela 1 — Cadastro com Validação

O artefato desta tela é a *Stored Procedure* SP_CADASTRAR_PARTICIPANTE_COMPLETO, que atende ao **RF2**. A procedure recebe os dados do participante (nome, e-mail e vínculo institucional) e de um projeto padrão, executando o seguinte fluxo: 1 comando SELECT para validar a unicidade do e-mail; 2 comandos INSERT para cadastrar o participante em TB_PARTICIPANTE e vinculá-lo ao projeto em RL_MEMBRO_PROJETO; e 2 comandos UPDATE para padronizar o nome do participante em caixa alta e atualizar os dados do projeto.

2.2.2. Tela 2 — Cadastro ou Validação com Transação

O artefato desta tela é a *Stored Procedure* SP_GERENCIAR_ATIVIDADE, que atende ao **RF1**. A procedure utiliza a lógica de Transação implícita do PostgreSQL (bloco BEGIN ... END), executando: 2 comandos SELECT para validar a existência do projeto e do instrutor; 1 comando INSERT em TB_ATIVIDADE; 2 comandos UPDATE para atualizar o conteúdo da atividade e o registro do instrutor; e 1 comando DELETE para remover eventuais atividades canceladas sem inscrições. Em caso de falha em qualquer etapa do fluxo, o ROLLBACK é acionado automaticamente, impedindo inserções parciais no banco.

2.2.3. Transações Explícitas

Além das *Stored Procedures*, foram desenvolvidas duas transações explícitas para operações que exigem controle de atomicidade e consistência em múltiplas tabelas simultaneamente.

SP_INSCREVER_COM_VALIDACAO atende ao **RF2**. Realiza a inscrição de um participante em uma atividade com validação de capacidade, garantindo que o processo ocorra de forma atômica — ou a inscrição é efetivada por completo ou nenhuma alteração é persistida no banco em caso de falha.

SP_TRANSFERIR_PARTICIPANTE atende ao **RF7**. Realiza a transferência de um participante entre projetos de extensão, atualizando os registros em **RL_MEMBRO_PROJETO** de forma transacional, assegurando que a remoção do vínculo anterior e a criação do novo vínculo ocorram como uma operação indivisível.

2.2.4. Dashboard 1 — Gráficos Analíticos Estratégicos

Foram desenvolvidas 4 *Materialized Views* para disponibilizar, de forma performática, os resultados de consultas avançadas exclusivas desta entrega, atendendo aos seguintes requisitos:

VM_INDICE_RETENCAO_PROJETO atende ao **RF2**, calculando a taxa de retenção de participantes que retornam a mais de uma atividade no mesmo projeto, por meio de CTEs e CASE com agregação condicional.

VM_GAP_EMISSAO_CERTIFICADO atende ao **RF3**, calculando o gap temporal médio entre a realização da atividade e a emissão do certificado, com função de janela AVG OVER por projeto.

VM_DISTRIBUICAO_QUARTIS_NOTAS atende ao **RF6**, segmentando a distribuição de notas em quartis via **NTILE (4)**, cruzada com o tipo de vínculo institucional dos participantes.

VM_COBERTURA_INSTRUTORES_PROJETO atende ao **RF1**, analisando a cobertura docente por projeto por meio da razão instrutor/atividade, carga média por instrutor e ranking de diversidade via **RANK**.

2.2.5. Dashboard 1 — Gráficos Analíticos Operacionais

Foram desenvolvidas 6 *Materialized Views* adicionais, originadas de 2 consultas avançadas e 4 consultas intermediárias exclusivas desta entrega, atendendo aos seguintes requisitos:

VM_COHORT_PRIMEIRA_INSCRICAO atende ao **RF2**, agrupando participantes pelo mês de primeira inscrição e calculando engajamento per capita por meio de CTEs encadeadas e **SUM OVER**.

VM_NPS_POR_PROJETO atende ao **RF4**, calculando o Net Promoter Score simplificado por projeto, classificando feedbacks em Detratores, Neutros e Promotores.

VM_MATRIZ_VINCULO_PRESENCA atende ao **RF2**, apresentando uma tabela cruzada entre tipo de vínculo institucional e status de presença, com taxas percentuais e nota média por segmento.

VM_IMPACTO_PATROCINIO_ATIVIDADE atende ao **RF5**, comparando o volume de inscrições e a nota média entre atividades com e sem patrocínio externo.

VM_VOLUME_SEMANAL_INSCRICOES atende ao **RF2**, apresentando a evolução semanal de inscrições com média móvel de 4 semanas via `AVG OVER` com `ROWS BETWEEN`.

VM_ROI_PARCEIROS atende ao **RF5**, calculando o retorno sobre investimento de cada parceiro por meio do custo por certificado e custo por inscrição, utilizando `LEFT JOINS` encadeados em 5 tabelas.

2.3. Rotinas

Os scripts DDL completos estão disponíveis nos arquivos `routines_and_transactions.sql` e `materialized_views_dashboards.sql` no repositório do projeto.

2.3.1. Transações Explícitas

As transações explícitas foram implementadas da seguinte forma:

SP_INSCREVER_COM_VALIDACAO transação para inscrição com *rollback* parcial via bloco `BEGIN ...EXCEPTION ...END` em caso de capacidade excedida ou duplicidade de inscrição.

SP_TRANSFERIR_PARTICIPANTE transação para transferência atômica de um participante entre projetos de extensão, com proteção contra estado inconsistente via *exception handler*.

Nota técnica — Transações no PL/pgSQL

No PostgreSQL, comandos de transação parcial (`SAVEPOINT`, `ROLLBACK TO SAVEPOINT`, `RELEASE SAVEPOINT`) não podem ser chamados diretamente dentro de blocos PL/pgSQL — tentativas resultam em `ERROR: unsupported transaction command in PL/pgSQL`. A técnica correta é utilizar blocos `BEGIN ...EXCEPTION ...END`, que criam *savepoints* implícitos gerenciados internamente pelo SGBD. Quando uma exceção é lançada dentro do sub-bloco, o PostgreSQL reverte automaticamente todas as alterações realizadas naquele escopo (*rollback* parcial) e desvia a execução para o *handler* `EXCEPTION`.

2.3.2. Demonstração de Uso

A seguir são apresentados exemplos de chamada das rotinas implementadas no PostgreSQL, bem como os comandos de atualização das *Materialized Views* para os dashboards.

```
-- Tela 1: cadastro completo de participante
CALL SP_CADASTRAR_PARTICIPANTE_COMPLETO (
```

```

        'Joao Silva', 'joao.silva@email.com', 'ALUNO', 1);

-- Tela 2: gerenciamento de atividade com transacao
CALL SP_GERENCIAR_ATIVIDADE(
    'Workshop de Python Avancado', '2024-05-20', 1, 2, 8);

-- Transacao 1: inscricao com controle de vagas
CALL SP_INSCREVER_COM_VALIDACAO(5, 2);

-- Transacao 2: transferencia entre projetos
CALL SP_TRANSFERIR_PARTICIPANTE(10, 1, 3);

-- Atualizacao das Materialized Views para os dashboards
REFRESH MATERIALIZED VIEW VM_INDICE_RETENCAO_PROJETO;
REFRESH MATERIALIZED VIEW VM_GAP_EMISSAO_CERTIFICADO;
REFRESH MATERIALIZED VIEW VM_DISTRIBUICAO_QUARTIS_NOTAS;
REFRESH MATERIALIZED VIEW VM_COBERTURA_INSTRUTORES_PROJETO;
REFRESH MATERIALIZED VIEW VM_COHORT_PRIMEIRA_INSCRICAO;
REFRESH MATERIALIZED VIEW VM_NPS_POR_PROJETO;
REFRESH MATERIALIZED VIEW VM_MATRIZ_VINCULO_PRESENCA;
REFRESH MATERIALIZED VIEW VM_IMPACTO_PATROCINIO_ATIVIDADE;
REFRESH MATERIALIZED VIEW VM_VOLUME_SEMANAL_INSCRICOES;
REFRESH MATERIALIZED VIEW VM_ROI_PARCEIROS;

```

2.4. Políticas

Foram implementadas políticas baseadas nos pilares Privacidade, Segurança (RBAC) e Disponibilidade (Backup). O código SQL completo encontra-se no arquivo `privacy_and_security.sql`.

2.4.1. Privacidade e Segurança (Data Masking e RBAC)

Para proteger dados sensíveis dos participantes em conformidade com a LGPD, foi criada a *view* segura `VW_PARTICIPANTE_SEGURO`, que mascara os endereços de e-mail dos participantes, impedindo o acesso direto a esse dado por usuários analíticos.

O controle de acesso foi implementado por meio de duas *roles*:

analista_dados *role* com acesso restrito, sem permissão de `SELECT` direto nas tabelas fonte (`TB_PARTICIPANTE` e `TB_REGISTRO_FEEDBACK`). O acesso é concedido exclusivamente às *Views* e *Materialized Views*, garantindo que usuários analíticos consumam os dashboards operacionais sem exposição direta às identidades e contatos dos participantes.

gestor_extensao *role* com privilégios completos na base de dados, destinada à administração do sistema de extensão.

Essa arquitetura garante que os dados analíticos sejam acessíveis para fins operacionais sem ferir as exigências da LGPD quanto à proteção de contatos diretos e identidades

abertas nos registros de feedback.

2.4.2. Política de Backup

Para mitigar o risco de perda de dados, foi desenvolvido o script Shell `backup_policy.sh`, que implementa uma rotina de backup automatizada por meio de dump lógico utilizando `pg_dump` com formato customizado (`-Fc`). O script contempla os seguintes mecanismos:

Registro de execuções logs das operações de backup são mantidos para rastreabilidade e auditoria das execuções.

Política de retenção backups com mais de 7 dias são removidos automaticamente por meio do comando `find` com a flag `-mtime`, evitando acúmulo desnecessário de arquivos.

Agendamento a recomendação para o ambiente de produção é o agendamento do script via `crontab` com a expressão `0 2 * * *`, executando automaticamente toda madrugada às 02h00.

A combinação do dump lógico com formato customizado, política de retenção e agendamento noturno garante disponibilidade dos dados com impacto mínimo sobre a operação do sistema durante o horário de uso.

2.4.3. Governança das Materialized Views

Por serem estáticas, as *Materialized Views* exigem atualização periódica para refletir o estado atual dos dados. Foi definida a seguinte política de atualização via `REFRESH MATERIALIZED VIEW`, baseada na criticidade e na frequência de alteração dos dados de cada visão:

Atualização diária recomendada para *views* operacionais com alta frequência de alteração nos dados base:

- VM_VOLUME_SEMANAL_INSCRICOES
- VM_COHORT_PRIMEIRA_INSCRICAO
- VM_MATRIZ_VINCULO_PRESENCA

Atualização semanal recomendada para *views* analíticas cujos dados base se alteram com menor frequência:

- VM_INDICE_RETENCAO_PROJETO
- VM_NPS_POR_PROJETO
- VM_IMPACTO_PATROCINIO_ATIVIDADE
- VM_ROI_PARCEIROS

Atualização sob demanda recomendada para *views* estratégicas, atualizadas após eventos específicos como encerramento de atividades ou emissão de certificados em lote:

- VM_GAP_EMITTASO_CERTIFICADO
- VM_DISTRIBUICAO_QUARTIS_NOTAS
- VM_COBERTURA_INSTRUTORES_PROJETO

A execução dos REFRESH pode ser integrada à mesma `crontab` utilizada para o backup, garantindo que os dashboards estejam sempre atualizados antes do início do expediente.

2.5. Conclusão

O presente relatório descreveu o desenvolvimento dos artefatos do Marco 2 (OCRA) do Sistema de Gestão de Extensão do Instituto de Computação da UFBA, expandindo o banco de dados relacional construído na Entrega 1 com novas funcionalidades voltadas ao suporte transacional, à análise de dados e à governança.

As *Stored Procedures* e transações explícitas implementadas (`SP_CADASTRAR_PARTICIPANTE_COMPLETO`, `SP_GERENCIAR_ATIVIDADE`, `SP_INSCREVER_COM_VALIDACAO` e `SP_TRANSFERIR_PARTICIPANTE`) encapsulam as operações críticas do sistema com controle transacional robusto, garantindo atomicidade e prevenindo estados inconsistentes no banco por meio do mecanismo de `BEGIN . . . EXCEPTION . . . END` do PL/pgSQL.

As 10 *Materialized Views* desenvolvidas para os dashboards analíticos estratégicos e operacionais introduzem consultas avançadas inéditas em relação à Entrega 1, cobrindo análises de retenção, cohort, NPS, ROI de parceiros e distribuição de desempenho. A definição de uma política de atualização periódica — diária, semanal ou sob demanda, conforme a criticidade de cada visão — garante que os dashboards reflitam o estado atual dos dados sem comprometer a performance do sistema.

As políticas de privacidade e segurança foram reforçadas com a implementação de *Data Masking* via `VW_PARTICIPANTE_SEGURO` e com a definição de duas *roles* com privilégios granulares (`analista_dados` e `gestor_extensao`), em conformidade com as exigências da LGPD. A política de backup foi consolidada com um script Shell automatizável via `crontab`, com política de retenção de 7 dias e registro de execuções para auditoria.

Por fim, as inconsistências identificadas na Entrega 1 foram corrigidas na versão revisada do relatório anterior, abrangendo o modelo lógico, o benchmark de desempenho, a nomenclatura conforme MAD1, as políticas de segurança e as consultas desenvolvidas.

2.6. Referências

Elmasri, R. and Navathe, S. B. (2018) “Sistemas de Banco de Dados”, 7. ed., Pearson.

DATASUS. “Metodologia de Administração de Dados (MAD)”, Ministério da Saúde. Disponível em: <https://datasus.saude.gov.br>.

ISO/IEC 11179-5 (2015) “Metadata Registries (MDR) — Part 5: Naming and Identification Principles”, Geneva: ISO.

Anexo: Scripts

Os seguintes scripts compõem os artefatos entregues neste marco:

routines_and_transactions.sql script contendo as *Stored Procedures* e transações explícitas desenvolvidas.

materialized_views_dashboards.sql script contendo as *Materialized Views* para os dashboards analíticos.

privacy_and_security.sql script contendo as políticas de privacidade, *Data Masking* e controle de acesso (RBAC).

backup_policy.sh script Shell com a rotina automatizada de backup.

Anexo: Tutorial de Reprodução

Para reproduzir o ambiente completo e testar as funcionalidades entregues neste marco, execute os scripts na seguinte ordem:

1. **final_script.sql**: cria o esquema de tabelas, insere a massa de dados inicial e cria as *views* e índices do Marco 1.

```
psql -U seu_usuario -d seu_banco -f final_script.sql
```

2. **routines_and_transactions.sql**: cria as *Stored Procedures* e transações, que dependem das tabelas criadas no passo anterior.

```
psql -U seu_usuario -d seu_banco -f  
routines_and_transactions.sql
```

3. **materialized_views_dashboards.sql**: após garantir as tabelas e rotinas, cria as *Materialized Views* para cache das consultas analíticas.

```
psql -U seu_usuario -d seu_banco -f  
materialized_views_dashboards.sql
```

4. **privacy_and_security.sql**: aplica as políticas de segurança, *views* de *Data Masking* e RBAC.

```
psql -U seu_usuario -d seu_banco -f privacy_and_security  
.sql
```

5. **benchmark.sql** (opcional): verifica a performance dos índices com o plano de otimização.

Anexo: Fundamentação Teórica e Testes Adicionais

As transações `SP_INSCREVER_COM_VALIDACAO` e `SP_TRANSFERIR_PARTICIPANTE` garantem a propriedade de Atomicidade (ACID): ou todas as operações do bloco são aplicadas com sucesso (COMMIT), ou, se qualquer violação for detectada, todas as alterações são revertidas integralmente pelo *savepoint* implícito do *handler* `EXCEPTION`.

A técnica utilizada são blocos `BEGIN ...EXCEPTION ...END` no PL/pgSQL, que criam *savepoints* implícitos gerenciados pelo PostgreSQL. Quando uma exceção é levantada dentro do sub-bloco, o SGBD reverte automaticamente as operações realizadas dentro daquele `BEGIN` interno e desvia para o *handler*. Isso difere de `SAVEPOINT/ROLLBACK TO SAVEPOINT` explícitos, que não são suportados diretamente em PL/pgSQL.

Demonstração de *rollback* por falta de vaga:

```
CALL SP_INSCREVER_COM_VALIDACAO(5, 2);
-- Output esperado (capacidade excedida):
-- NOTICE: Inscricao revertida: Capacidade excedida para a
--          atividade 2.
--          Limite: 100, Atual: 101
```

Demonstração de transferência bem-sucedida:

```
CALL SP_TRANSFERIR_PARTICIPANTE(10, 1, 3);
-- Output esperado:
-- NOTICE: Transferencia concluida: "PARTICIPANTE 10"
--          movido do projeto 1 para o projeto 3.
```

Justificativa para uso de Materialized Views

O uso de *Materialized Views* é justificado pela redução de carga computacional em tempo real para dashboards operacionais. Ao contrário de uma *View* comum, que executa o `SELECT` a cada chamada, as *Materialized Views* armazenam o resultado fisicamente, entregando tempo de resposta $O(1)$ para a camada de apresentação.

O *trade-off* é a necessidade de atualização programada via `REFRESH MATERIALIZED VIEW`, o que é plenamente aceitável em cenários de *Business Intelligence* que não exigem precisão de segundos (ex: atualização diária durante a madrugada).